Sauron

O Sauron organiza a fila e a alocação de GPUs do cluster. Você leva seu código para a máquina, descreve o job num arquivo, e o Sauron roda seu container numa GPU livre e avisa quando termina.

Primeiros passos

1. Entre por SSH na sessão de login (Acesso).
2. Peça a um administrador que registre você no Sauron. Sem isso os comandos respondem que você não tem acesso.
3. Rode `sauron whoami`. Se aparecem seu usuário e sua cota, o acesso está pronto.
4. Clone o projeto em `~/workspace/meu-projeto` (Levar o código).
5. Baixe o kit para a pasta do projeto e ajuste o `job.yaml` (com `workspace: meu-projeto`) e o `script.sh` (Descrever o job).
6. Envie primeiro uma versão curta, com poucos passos e `max_time` de minutos: `sauron submit -c job.yaml script.sh`.
7. O envio mostra o número do job (ex.: `Job #42`). Acompanhe com `sauron logs 42 -f`. Deu certo? Envie o job completo.

> **Não faça**
>
> - Rodar treino, inferência ou notebook na sessão de login. Ela não tem GPU; tudo isso vai por job.
> - Deixar código, dados ou modelos na home (`~`). Use `~/workspace`.
> - Gravar checkpoints e resultados fora de `~/workspace`. O container é apagado quando o job termina, e o que foi gravado fora dela some junto.
> - Pôr token ou senha no `job.yaml` ou no comando. Use um `.env` (Segredos).
> - Pedir mais `vram`, `memory` ou `max_time` do que precisa. Seu job espera mais e segura recurso de quem está na fila.
> - Guardar no cluster a única cópia do que importa. Não há backup (Armazenamento).

Acesso e sessão de login

Entre por SSH com o seu usuário, no endereço que o administrador te enviou. A senha é a mesma que você já usa nas outras máquinas do laboratório:

```
ssh <seu-usuario>@<endereço-do-login>
```

Ao conectar você cai na sua sessão de login. Ela serve para preparar código, editar arquivos, enviar jobs e acompanhar resultados, e não tem GPU nem acesso a Docker ou `nvidia-smi`.

Levar o código

Na sessão de login, clone seu projeto numa subpasta de `~/workspace`, o disco rápido que o job enxerga. Não deixe código, dados ou modelos direto na home (`~`): ela é pequena e lenta.

```
cd ~/workspace
git clone <url-do-seu-repo> meu-projeto
```

Descrever o job

Um job tem dois arquivos, os dois na pasta do projeto: o `job.yaml` diz do que o job precisa, e o `script.sh` diz o que ele roda. Os dois vêm no kit.

`job.yaml`:

```
name: meu-treino
image: pytorch/pytorch:2.13.0-cuda13.0-cudnn9-devel
gpus: 1
vram: 80                  # VRAM mínima por GPU, em GB
max_time: 8h

workspace: meu-projeto    # subpasta de ~/workspace; o job já começa dentro dela
```

`script.sh`, com os comandos em bash, um por linha:

```
set -e
python train.py
```

Envie com:

```
sauron submit -c job.yaml script.sh
```

Para um comando só, dá para usar um arquivo único: ponha `command: python train.py` no `job.yaml` e envie com `sauron submit -f job.yaml`. O `sauron new` monta esse arquivo fazendo perguntas no terminal; em scripts, use `sauron submit -f`.

Campos opcionais: `cpus`, `memory` (ex.: `8g`), `env` (só variáveis *não* secretas) e `volumes` (montar dados extras). Uma chave escrita errada gera erro no envio, então não passa em branco.

`vram` escolhe a classe de GPU pelo mínimo de VRAM, não uma fração. As classes disponíveis aparecem ao vivo no `sauron new` e no `sauron status`: peça a menor que te atende. Sua cota é em GB de VRAM: o `sauron whoami` mostra sua cota e o que resta.

O job gasta da cota o tamanho da placa onde roda, não o número que você pôs em `vram`, e o gasto vale para cada GPU pedida. Com classes de 89 GB e 179 GB, por exemplo:

- `gpus: 1` e `vram: 20`: roda numa placa de 89 GB e gasta 89 GB da cota.
- `gpus: 1` e `vram: 100`: só cabe na de 179 GB e gasta 179 GB.

Com mais de 1 GPU, o job pode precisar de uma classe maior do que com 1 GPU. Se o gasto passar do que você tem livre, o `sauron submit` avisa quanto o job vai gastar.

A cota volta quando o job termina. Se os jobs que você já tem rodando ocupam a cota, o próximo espera na fila, e o `sauron status <ref>` mostra esse motivo.

Vários jobs rodam na mesma máquina ao mesmo tempo e dividem a mesma RAM. `memory` é o teto de RAM do seu container. Se o job passar do teto, só ele é encerrado por falta de memória, não a máquina. Declare um valor realista com `memory:` (ex.: `16g`): pedir demais tira memória de quem também precisa, pedir de menos derruba o seu job.

Quando o `sauron status` mostra "disponível até 64 GB de RAM", um job com `memory:` de até `64g` começa na hora.

Imagem e dependências

O campo `image` diz em qual container o seu job roda. O Sauron baixa essa imagem pronta de um registry, como o Docker Hub ou o NGC da NVIDIA, e roda o seu comando dentro dela. Você não precisa de Dockerfile. Na primeira vez em que uma imagem é usada, a máquina baixa a imagem antes de começar o job; acompanhe o download pelo `sauron logs`.

Caminho recomendado: imagem pronta + suas dependências no script

Escolha uma imagem que já traga CUDA e o framework que você usa:

```
image: pytorch/pytorch:2.13.0-cuda13.0-cudnn9-devel
```

Instale o resto num venv, no início do job, pelo `script.sh`:

```
set -e
export PYTHONNOUSERSITE=1
python -m venv --without-pip --system-site-packages .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python train.py
```

O venv fica na pasta do projeto, aproveita os pacotes da imagem e vale para os próximos jobs; para começar do zero, apague a pasta `.venv`. O `PYTHONNOUSERSITE=1` impede que pacotes antigos de `~/workspace/.local` passem na frente dos do venv.

Downloads do pip, do Hugging Face e do PyTorch vão para um cache da máquina, compartilhado entre os jobs: o que foi baixado uma vez não é baixado de novo. O job já vem com `HF_HOME` e `TORCH_HOME` apontando para ele; não troque essas variáveis e não baixe modelos para a pasta do projeto.

Imagem própria

Se o ambiente é grande ou demora para instalar, monte uma imagem a partir de uma das acima no seu computador, publique num registry público (Docker Hub, GitHub Container Registry) e use o nome no `job.yaml`:

```
image: seu-usuario/meu-ambiente:v3
```

Use sempre uma versão fixa e troque o número a cada mudança: a máquina guarda a imagem depois do primeiro uso, e só um nome novo garante que o job roda a versão que você acabou de publicar.

Enviar e acompanhar

`sauron submit -c job.yaml script.sh`	envia o job
`sauron list`	lista seus jobs mais recentes
`sauron status`	tempo de espera por tamanho de GPU
`sauron status <ref>`	estado do job, posição na fila e motivo da espera
`sauron logs <ref>`	mostra as últimas 100 linhas dos logs
`sauron logs <ref> -f`	acompanha os logs em tempo real, começando pelas últimas 100 linhas
`sauron logs <ref> -n N`	mostra só as últimas N linhas (`-n all` para tudo); combina com `-f`
`sauron logs <ref> --head N`	mostra só as primeiras N linhas
`sauron queue`	mostra a fila e a estimativa de espera
`sauron whoami`	mostra sua cota, o que resta e seus grupos

`list`, `status`, `queue` e `whoami` aceitam `--json`, para usar em scripts.

A referência `<ref>` é o número do job, mostrado no `list`. Para controlar um job:

`sauron cancel <ref>`	encerra dando tempo para finalizar
`sauron kill <ref>`	encerra na hora
`sauron retry <ref>`	recoloca na fila um job que falhou, dentro do prazo mostrado no `sauron status <ref>`

Segredos

> **Segredos só no .env**
>
> Segredos nunca vão no `job.yaml` nem no comando, o envio recusa tokens e chaves. Grave um arquivo `.env` na pasta do projeto e carregue dentro do job:

```
source .env
```

Notificações no Telegram

Receba aviso quando o job começa, termina ou falha:

```
sauron telegram connect
```

Escaneie o QR que aparece no terminal (link válido por 15 min) e toque em START. O aviso de falha indica como ver o log. Confira o estado com `sauron telegram show`.

Com o chat vinculado, mande `/status` pro bot para ver a disponibilidade do cluster na hora: o tempo de espera por tamanho de GPU. Com `/status 42` você vê o estado de um job seu, e `/jobs` lista seus jobs ativos e os últimos terminados.

Mande `/logs` para ver o final do log do seu job (`/logs 42` escolhe o job). Com `/autologs` o log passa a vir dentro do próprio aviso de falha; mande de novo para desligar.

Armazenamento e backup

Sua pasta na máquina é espaço de trabalho, não repositório permanente. Limpe com regularidade o que não usa mais: arquivos temporários, checkpoints antigos e venvs que não usa.

O cluster não faz backup automático, e uma falha de hardware pode levar tudo. Mantenha fora do cluster a cópia principal de código, datasets, checkpoints e resultados importantes.

Boas práticas

- Dê folga ao `max_time`, mas sem exagero: o job é encerrado ao passar do limite, e o valor entra na estimativa de espera de todo mundo.
- Classe de `vram` menor costuma liberar antes, e o uso recente de GPU pesa no desempate da fila.
- Salve checkpoints periodicamente. Se o job falhar ou estourar o tempo, o `sauron retry` recomeça do zero a não ser que seu código retome do último checkpoint. O retry vale por pouco tempo depois da falha (em geral 15 minutos); passado o prazo, envie o job de novo com `sauron submit`.

vLLM

Para rodar modelos de linguagem com o vLLM, use a imagem oficial `vllm/vllm-openai`. Ela já traz o vLLM, o CUDA e o pacote `openai`. Baixe o vllm.yaml e o vllm.sh para a pasta do projeto e envie com:

```
sauron submit -c vllm.yaml vllm.sh
```

Com essa imagem use sempre o `-c`, mesmo para um comando só.

Gerar em lote

Para gerar respostas para muitos prompts de uma vez, chame o vLLM direto do seu código Python:

```
from vllm import LLM, SamplingParams

llm = LLM("Qwen/Qwen2.5-7B-Instruct")
prompts = ["Explique o que é uma GPU em uma frase.", "Dê um nome para um gato."]
saidas = llm.generate(prompts, SamplingParams(max_tokens=100))
for saida in saidas:
    print(saida.outputs[0].text)
```

Nesse caso o `vllm.sh` tem uma linha só: `python3 gerar.py`. Na imagem do vLLM o comando é `python3`.

Servidor e o seu código no mesmo job

Se o seu código conversa com uma API no formato da OpenAI, suba o servidor do vLLM no próprio job. O `vllm.sh` do kit faz isso: sobe o servidor, espera o modelo carregar e roda o seu código. Se o servidor cair, o job mostra o motivo e encerra.

```
set -e

MODEL=Qwen/Qwen2.5-7B-Instruct

vllm serve "$MODEL" --port 8000 > vllm.log 2>&1 &
servidor=$!
trap 'kill $servidor 2>/dev/null' EXIT

until curl -sf localhost:8000/health > /dev/null; do
  kill -0 $servidor 2>/dev/null || { tail -50 vllm.log; exit 1; }
  sleep 5
done

python3 cliente.py
```

No seu código, aponte o cliente para `localhost:8000`:

```
from openai import OpenAI

cliente = OpenAI(base_url="http://localhost:8000/v1", api_key="local")
resposta = cliente.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[{"role": "user", "content": "Explique o que é uma GPU em uma frase."}]
)
print(resposta.choices[0].message.content)
```

As mensagens do servidor ficam em `vllm.log`, na pasta do projeto.

Ajuda e feedback

`sauron guide`	mostra o guia rápido no terminal
`sauron --help`	lista todos os comandos
`sauron <comando> --help`	ajuda de um comando
`sauron feedback "sua mensagem"`	manda um problema ou sugestão aos mantenedores

Atualizado em 17/10/2026